

Documentación Completa - Terminal Souls

El programa comienza con:

`import random` Aquí estás importando una librería de Python que sirve para generar números aleatorios. La usas para calcular daño, probabilidades de crítico y ataques especiales.

Luego defines la función del menú:

`def menu()`: Defines una función llamada menu que se encarga de mostrar el menú principal del juego.

`opcion = ""` Creas una variable vacía donde vas a guardar lo que el usuario elija.

`while opcion != "1" and opcion != "2":` Este bucle se repite mientras el usuario NO elija una opción válida. Solo se acepta "1" o "2". Esto es validación de entrada.

`Los print() siguientes:` Muestran el título del juego y las opciones disponibles. No afectan la lógica, solo la interfaz visual.

`opcion = input("Choose an option: ")` Le pides al usuario que escriba una opción y la guardas.

`if opcion != "1" and opcion != "2":` Si el usuario escribe algo diferente, entra aquí.

`print("Invalid option, try again.\n")` Muestra error y vuelve al inicio del while.

`return opcion` Cuando el usuario por fin elige bien, devuelves esa opción al programa principal.

`def generar_damage(min, max):` Defines una función para generar daño.

`return random.randint(min, max)` Devuelve un número aleatorio entre esos dos valores. Esto hace que el juego no sea predecible.

`def status(hp_heroe, hp_enemy):` Función que muestra la vida de ambos personajes.

`print("\nHERO health:", hp_heroe, "hp")` Imprime la vida del héroe.

`print("ENEMY health:", hp_enemy, "hp")` Imprime la vida del enemigo.

Esto se ejecuta cada turno para ver el estado del juego.

`def critical_hit(damage):` Función que determina si hay golpe crítico.

`prob = random.randint(1, 100)` Genera un número entre 1 y 100.

`if prob <= 10:` Si el número es menor o igual a 10 → hay crítico (10% probabilidad).

`return damage * 2` Duplica el daño.

`return damage` Si no hay crítico, devuelve el daño normal.

`def turn_player()` Función que controla TODO el turno del jugador.

`global hp_heroe, hp_enemy, potion` Permite modificar variables que están fuera de la función. Sin esto, no podrías cambiar la vida ni las pociones.

`invalid_option = False` Variable que controla si la opción es válida o no.

while not invalid_option:Se repite hasta que el jugador elija bien.

Los print() muestran el menú de acciones del jugador.

option = input("Choose: ")Captura la acción del jugador.

ATAQUE NORMAL

if option == "1":Si elige atacar.

damage = generar_damage(10, 25)Genera daño entre 10 y 25.

damage = critical_hit(damage)Aplica posible crítico.

hp_enemy -= damageResta la vida del enemigo.

hp_enemy = max(hp_enemy, 0)Evita que la vida quede negativa.

invalid_option = TrueTermina el turno.

POCIÓN

elif option == "2":Si elige curarse.

if hp_heroe == 100:Si ya tiene vida completa.

No lo deja usar poción.

elif potion > 0:Si tiene pociones disponibles.

hp_heroe += 20Recupera vida.

hp_heroe = min(hp_heroe, 100)Evita pasar de 100.

potion -= 1Reduce la cantidad de pociones.

invalid_option = TrueTermina turno.

else:Si no tiene pociones.

Muestra error y repite turno.

ATAQUE ESPECIAL

elif option == "3":

probability = random.randint(1, 100)Define si el ataque funciona.

if probability <= 50:50% de éxito.

hp_enemy -= damageHace daño fuerte.

else:Si falla, no hace nada.

`invalid_option = True` Termina turno igual.

OPCIÓN INVÁLIDA

`else`: Si escribe algo incorrecto.

Muestra error y NO termina el turno.

`def IA_enemy hp_enemy`: Función que decide si el enemigo se cura.

`return hp_enemy <= 24` Si la vida es baja, devuelve True.

`def turn_enemy()`: Controla el turno del enemigo.

`if IA_enemy hp_enemy`: Si la IA dice que se cure.

`hp_enemy += 20` Se cura.

`hp_enemy = min hp_enemy, 120` No pasa de su vida máxima.

`else`: Si no se cura.

`damage = generar_damage 15, 20` Genera daño.

`hp_heroe -= damage` Ataca al jugador.

`hp_heroe = max hp_heroe, 0` Evita negativos.

`def who_won()`: Función que verifica si alguien ganó.

`if hp_enemy <= 0`: Si el enemigo murió → ganas.

`elif hp_heroe <= 0`: Si el héroe murió → pierdes.

`return False` Si nadie ha muerto, el juego sigue.

`jugar = "yes"` Variable que controla si el juego se repite.

`while jugar == "yes"`: Bucle principal del juego.

`opcion_menu = menu()` Llama al menú.

`if opcion_menu == "2"`: Si elige salir.

`break` Termina el programa.

REINICIO DE VARIABLES

Se reinician todas las variables del juego:

- vida del héroe
- vida del enemigo
- pociones
- turno

Esto permite jugar otra vez desde cero.

BUCLE DE COMBATE

`while hp_heroe > 0 and hp_enemy > 0`:El juego continúa mientras ambos estén vivos.

`if turn_heroe`:Si es turno del jugador.

`turn_player()`Ejecuta su turno.

`turn_heroe = False`Cambia al enemigo.

`else`:Turno del enemigo.

`turn_enemy()`Ejecuta su turno.

`turn_heroe = True`Vuelve al jugador.

`if who_won()`:Si alguien ganó.

`break`Sale del combate.

`print("\nEND GAME")`Muestra que el juego terminó.

`jugar = input("Do you want to play again? (yes/no): ").lower()`Pregunta si quiere volver a jugar.

`.lower()`Convierte a minúsculas para evitar errores.

`while jugar != "yes" and jugar != "no"`:Valida la respuesta.

Si escribe algo raro, vuelve a preguntar.

`print("\nThanks for playing!!!")`Mensaje final cuando el usuario decide salir.

CONCLUSIÓN (modo roomie)

Este código:

- Usa funciones para organizar
- Usa `while` para controlar flujo
- Usa `if` para decisiones
- Maneja estados del juego
- Tiene validación real
- Simula un combate completo